International University of Technology
Twintech

Visual Programming

C#

By.
Mr. Mohammed Talat

LAB EVALUATION

Student Name:----------

Department:---------- Group Number:----------

ID	Lab Topics	Date of Conducted	Programs / Activity	Assignments	Signature
1		/ /			
2		/ /			
3		/ /			
4		/ /			
5		/ /			
6		/ /			
7		/ /			
8	**Building Controllers, Configuring Dependency Injection, & Database Migrations in Web API (.NET 5)**	/ /			
9		/ /			
10		/ /			

Student Responsibilities:

1) **Lab Sheet Sign-off:** Students are required to obtain a signature on all lab sheets before leaving the lab. Failure to do so will result in a **zero** grade for the respective lab.
2) **Independent Work:** All program outputs and assignments must be completed independently by each student.
3) Students are responsible for maintaining the lab manual in a clean, safe, and organized condition.
4) **Lab Manual Submission:** The entire lab manual must be submitted in person during Week 11.- *any student failed to do so will get ZERO in the final mark.*

International University of Technology
Twintech

Building Controllers, Configuring Dependency Injection, & Database Migrations in Web API (.NET 5)

" Bugs are just features waiting to be fixed. Keep going!"

1.0 Objectives

After completing this lab, students will be able to:

- **Understand the role** of the Web API Layer as the Presentation boundary in Clean Architecture.
- **Configure standard** Dependency Injection (DI) lifetimes for Repositories and Services in .NET
- **Create a reusable** base controller to unify HTTP logging and API response structures.
- **Implement RESTful** endpoints (GET, POST, PUT) to expose application services safely.
- **Perform Entity Framework Core** database migrations using the Package Manager Console.

1.1 Introduction

In this lab, we advance from project initialization to bringing our **Web API Layer** to life. We will bridge the gap between HTTP requests sent by external clients and the core business logic residing in our internal layers.

Students will learn how to configure the central entry point of a .NET 5 application using the Program.cs file to set up essential pipeline middlewares, routing, and **Dependency Injection**. To ensure clean and production-ready code, we will implement a centralized **BaseController** that handles structured JSON responses and error logging globally. Finally, students will bridge their entities to a real database by configuring a connection string and executing **EF Core Migrations** to generate the database schema automatically.

2.0 Configuring Startup & Dependency Injection in .NET 5

In .NET 5, configuring the service container (Dependency Injection) and building the HTTP request pipeline is essential for managing object lifetimes across our architecture.

2.1 Installing Required Infrastructure Packages

To allow our Web API project to communicate with SQL Server and resolve our AutoMapper profiles, we must install the appropriate ecosystem packages. Open your **NuGet Package Manager or NuGet Package Manager Console**, target the **Web API Project**, and run:

```
# Install Entity Framework Core SQL Server support
add-package Microsoft.EntityFrameworkCore.SqlServer --version 5.0.17

# Install AutoMapper Dependency Injection extensions
add-package AutoMapper.Extensions.Microsoft.DependencyInjection --version 8.1.1
```

2.2 Structuring the Program.cs File

Unlike modern .NET versions that use top-level statements, .NET 5 uses the explicit Host.CreateDefaultBuilder pattern. Open your **Program.cs** file in the Web API project and replace its contents with the configuration below to bind your DbContext, AutoMapper, and application services:

```
using System;
using System.Collections.Generic;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.OpenApi.Models;
using Application.IService;
using Application.ServiceImpl;
using Domain.IRepository;
using Infrastructure.Data;
using Infrastructure.Repository;

namespace UserManagement
{
    public class Program
    {
        public static void Main(string[] args)
        {
            CreateHostBuilder(args).Build().Run();
        }

        public static IHostBuilder CreateHostBuilder(string[] args) =>
            Host.CreateDefaultBuilder(args)
                .ConfigureWebHostDefaults(webBuilder =>
                {
                    webBuilder.ConfigureServices((context, services) =>
                    {
                        var configuration = context.Configuration;

                        // 1. DbContext
                        services.AddDbContext<AppDbContext>(options =>
                            options.UseSqlServer(configuration.GetConnectionString("DefaultConnection")));

                        // 2. DI Repositories
                        services.AddScoped<IUserRepository, UserRepository>();

                        // 3. DI Services
                        services.AddScoped<IUserService, UserServiceImpl>();

                        // 4. AutoMapper
                        services.AddAutoMapper(AppDomain.CurrentDomain.GetAssemblies());

                        // 5. Controllers
                        services.AddControllers();

                        // 6. Swagger Config
                        services.AddSwaggerGen(c =>
                        {
                            c.SwaggerDoc("v1", new OpenApiInfo { Title = "User Management API", Version = "v1" });
                        });
                    });

                    webBuilder.Configure((context, app) =>
                    {
                        var env = context.HostingEnvironment;

                        // Configure pipeline
                        if (env.IsDevelopment())
                        {
                            app.UseDeveloperExceptionPage();
                        }

                        // Swagger Middleware
                        app.UseSwagger();
                        app.UseSwaggerUI(c =>
                        {
                            c.SwaggerEndpoint("/swagger/v1/swagger.json", "User Management API v1");
                        });

                        app.UseHttpsRedirection();

                        // Required in .NET 5 for Routing and Endpoints
                        app.UseRouting();

                        app.UseEndpoints(endpoints =>
                        {
                            endpoints.MapControllers();
                        });
                    });
                });
    }
}
```

3.0 Implementing Unification: The Base Controller

To promote code reuse and strict response formatting, we define a centralized BaseController.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using System;

namespace ReservePro.Management.Api.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   public abstract class BaseController : ControllerBase
   {using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using System;

namespace ReservePro.Management.Api.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   public abstract class BaseController : ControllerBase
   {
      protected readonly ILogger<BaseController> Logger;

      protected BaseController(ILogger<BaseController> logger)
      {
         Logger = logger;
      }

      protected IActionResult HandleResponse<T>(T result, string message = null)
      {
         if (result == null)
         {
            Logger.LogWarning("Requested resource not found.");
            return NotFound(new
            {
               message = "Resource not found.",
               success = false
            });
         }

         var msgProp = result.GetType().GetProperty("Message");
         if (msgProp != null && message != null)
         {
            var currentValue = msgProp.GetValue(result);
            if (currentValue == null)
               msgProp.SetValue(result, message);
         }

         return Ok(result);
      }

      protected IActionResult HandleError(Exception ex, string message = "An unexpected error occurred.")
      {
         Logger.LogError(ex, message);

         return StatusCode(500, new
         {
            message = message,
            success = false,
            error = ex.Message
         });
      }
   }
}

      protected readonly ILogger<BaseController> Logger;

      protected BaseController(ILogger<BaseController> logger)
      {
         Logger = logger;
      }

      protected IActionResult HandleResponse<T>(T result, string message = null)
      {
         if (result == null)
         {
            Logger.LogWarning("Requested resource not found.");
            return NotFound(new { message = "Resource not found.", success = false });
         }
         return Ok(result);
      }

      protected IActionResult HandleError(Exception ex, string message = "An unexpected error occurred.")
      {
         Logger.LogError(ex, message);
         return StatusCode(500, new { message = message, success = false, error = ex.Message });
      }}}
```

4.0 Building RESTful Endpoints (The UserController)

Now we will create the concrete **UserController** that inherits from our BaseController. It injects IUserService using Constructor Injection to manage users via proper RESTful verbs (GET, POST, PUT). Inside the **Controllers** folder, create a new class named **UserController.cs** with the following implementation:

```
using System;
using System.Collections.Generic;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using Application.DTOs;
using Application.IService;
using UserManagement.Api.Controllers;

namespace UserManagement.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class UserController : BaseController
    {
        private readonly IUserService _userService;

        public UserController(ILogger<UserController> logger, IUserService userService)
            : base(logger)
        {
            _userService = userService;
        }


        [HttpPost]
        public IActionResult Create([FromBody] UserDto userDto)
        {
            try
            {
                if (userDto == null)
                {
                    return BadRequest(new { message = "Invalid user data.", success = false });
                }

                _userService.CreateUser(userDto);
                return HandleResponse(new { message = "User created successfully.", success = true });
            }
            catch (Exception ex)
            {
                return HandleError(ex, "Failed to create user.");
            }
        }

    }
}
```

5.0 Database Migrations Configuration

To configure your physical database connection, update your **appsettings.json** inside the Web API layer with your local SQL Server instance details:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=DESKTOP-06HOR2V\\SQLEXPRESS;Database=MYUserManagementDB;User
Id=sa;Password=Test123;TrustServerCertificate=True;"
  }
}
```

5.1 Executing PMC Migration Commands

1. Open the **Package Manager Console**.
2. Set the **Default project** dropdown menu to **Infrastructure**.
3. Set the **UserManagement.API** project as your startup project in the Solution Explorer.
4. Execute the following sequential commands to build the physical schema:

```
# Step A: Generate the local migration file
Add-Migration InitialCreate -StartupProject UserManagement.API

# Step B: Apply changes directly to your SQL Server instance
Update-Database -StartupProject UserManagement.API
```

5.0 Lab Assignment:

To reinforce today's concepts, complete the following architectural tasks for your Final Course Project:

1. Update your Web API project's configuration to implement the explicit Host.CreateDefaultBuilder model matching the .NET 5 runtime setup.
2. Complete the **Dependency Injection** configuration inside your service container for all remaining interfaces and implementations in your solution.
3. Build a fully functional **Controller** for your core domain entities ensuring it inherits from a generic BaseController and manages runtime exceptions correctly.
4. Configure your application's connection string, fix assembly version matching, and perform an **EF Core Migration** to completely initialize your database.